

Finding Similar Items in the New York Times Articles Dataset

A From-Scratch Pipeline Based on Shingling, MinHash and Locality-Sensitive Hashing

Tharun Reddy Banda

Algorithms for Massive Datasets — Master in Computer Science

University of Milan

Abstract

Near-duplicate detection over large text corpora cannot rely on exhaustive pairwise comparison, whose cost grows quadratically with the number of documents. This report presents a complete, from-scratch pipeline for detecting pairs of similar textual documents in the abstracts of the New York Times Articles & Comments (2020) dataset, combining three techniques studied in the course: k -shingling with a compact hashed representation, MinHash signatures built from a universal hash family, and Locality-Sensitive Hashing (LSH) with the banding technique, followed by exact verification. Exact Jaccard similarity is used both as the theoretical target that MinHash is designed to estimate and as the ground truth against which the whole pipeline is validated. The report describes the exact version of the dataset used, the way the data are organised and pre-processed, the algorithms and their implementation together with their time/space complexity and correctness arguments, an empirical and theoretical scalability analysis, a description of the experiments performed, and a discussion of the results, including the precision/recall trade-off induced by the choice of LSH parameters. On the pre-processed collection of 4,994 abstracts, the pipeline reduces the number of pairs that require exact comparison by more than four orders of magnitude with respect to the exhaustive baseline, while matching the exhaustive method's precision and recall on the evaluation subset for a well-chosen banding configuration.

Contents

1	Introduction	2
2	Dataset	2
3	Data organisation	3
4	Pre-processing	3
5	Algorithms and implementation	4
5.1	Exact Jaccard similarity	4
5.2	MinHash	5
5.3	Locality-Sensitive Hashing (banding)	6
6	Scalability of the proposed solution	7
7	Experiments	8
7.1	Accuracy of the MinHash estimator	8
7.2	End-to-end detection on the full collection	9
7.3	LSH quality against exact ground truth, and effect of (b, r)	10
8	Discussion of results	10

9	Replicability	12
10	Conclusions	13

1 Introduction

The task addressed in this project is to find, within a large collection of short text documents, all the pairs whose textual content is similar above a given threshold, without paying the quadratic cost of comparing every pair exhaustively. This is the classical *finding similar items* problem, and the report follows the standard three-stage solution presented in the course:

pre-processing → shingling → MinHash → LSH → candidate verification,

with exact Jaccard similarity used as the correctness reference throughout. Every core similarity-detection component — shingling, the MinHash signature scheme, and LSH banding — is implemented directly in Python, without relying on any external similarity/near-duplicate-detection library; only general-purpose libraries (NumPy, pandas, Matplotlib) are used for data handling, numerical vectorisation and plotting.

The remainder of the report is organised so as to directly address the seven points requested in the project specification: the dataset version and scope (Section 2), the data organisation (Section 3), the pre-processing pipeline (Section 4), the algorithms and their implementation, including their correctness and complexity (Section 5), the scalability of the proposed solution (Section 6), the experiments (Section 7), and a discussion of the results (Section 8). Section 9 additionally documents the exact environment and configuration needed to replicate every number reported in this document, and Section 10 concludes.

2 Dataset

Source and version. The project uses the *New York Times Articles & Comments (2020)* dataset, published on Kaggle under the CC BY-NC-SA 4.0 license, with identifier `benjaminawd/new-york-times`. As required by the project specification, the dataset is *not* stored in the code repository: it is downloaded programmatically at the beginning of the notebook, at every run, via the official Kaggle API (`kaggle datasets download`). Since the Kaggle dataset can in principle be updated by its maintainer, all figures reported in this document refer specifically to the copy of the dataset downloaded on **[TODO: insert the calendar date on which the notebook was executed to produce the results reported here, e.g. “12 September 2026”]** — this is the access date that should be considered as the reference version of the data for reproducibility purposes.

Parts of the dataset considered. The Kaggle archive bundles several files: a number of per-month comment files (`nyt-comments-part0.csv`, ...), a `train.csv/test.csv` split, and one article-level file. This project uses *only* the article-level file, `nyt-articles-2020.csv`, which contains 16,787 articles described by 11 columns: `newsdesk`, `section`, `subsection`, `material`, `headline`, `abstract`, `keywords`, `word_count`, `pub_date`, `n_comments` and `uniqueID`. The per-article *comments* files were not used in the experiments reported here; the pipeline is agnostic to the choice of text field, and could be pointed at concatenated user comments instead of abstracts by changing a single configuration variable (`TEXT_COLUMN`), with no change to the similarity-detection logic.

Within the article file, the `abstract` field was selected as the unit of comparison. Abstracts average about 127 characters (versus 53 characters for `headline`), which is long enough to yield a non-trivial number of word shingles per document while keeping the document-level vocabulary

compact. The **abstract** column has 3 missing values out of 16,787 rows; these, together with any whitespace-only abstracts, are dropped during pre-processing (Section 4).

Sub-sampling. Running the full pipeline on all 16,787 articles is possible with the implemented code (the algorithms scale as described in Section 6), but a fixed-size random subsample is used for the experiments reported here, to keep the notebook’s end-to-end running time short and grading-friendly, as suggested by the project specification. A single global flag, `USE_SUBSAMPLE`, switches between the full corpus and a subsample of `N_DOCUMENTS = 5,000` articles drawn with a fixed random seed (42); no other part of the pipeline needs to change when this flag is toggled.

3 Data organisation

The raw CSV file is loaded once into a single pandas `DataFrame` (`articles_df`). A dedicated, minimal `documents` table is then derived from it, containing only the selected text column, coerced to string and stripped of missing or empty entries; if the requested column is not found in the source file, the notebook raises an explicit error rather than silently falling back to a different field, so that a mis-configuration cannot pass unnoticed.

If sub-sampling is enabled, `N_DOCUMENTS` rows are drawn from this table with a fixed random seed. Every subsequent processing stage augments the same `documents` table with one additional column, so that the whole pipeline operates on a single, consistent, row-aligned data structure and every intermediate representation of a document (raw text, cleaned text, shingle set, MinHash signature) remains addressable by the same row index throughout the notebook:

Column	Content
<code>text</code>	raw abstract text
<code>clean_text</code>	text after pre-processing (Section 4)
<code>shingles</code>	set of hashed k -shingles

MinHash signatures are additionally stored as the rows of a single dense NumPy matrix of shape $n \times h$ (documents $\times$ hash functions), rather than as a further `DataFrame` column, since this is the representation required by both the MinHash-accuracy check and the LSH banding step (Section 5).

For the results reported in this document, 5,000 abstracts are sampled with random seed 42; after discarding documents that are empty after cleaning or that are too short to produce a shingle (see below), the working collection consists of 4,994 documents. This is the collection used, unless stated otherwise, throughout Sections 5–7.

4 Pre-processing

The pre-processing pipeline is intentionally simple, so that the behaviour of the similarity-detection algorithms is not confounded by an elaborate, hard-to-audit NLP pipeline (e.g. stemming, stop-word removal, or language detection). Each abstract undergoes the following steps, in order:

- (a) **Normalisation:** the text is lower-cased.
- (b) **Tag removal:** HTML-like tags matching the pattern `<[>]+>` are removed.
- (c) **Character filtering:** every character that is not a lowercase letter or whitespace is removed (digits and punctuation are discarded).

- (d) **Whitespace normalisation:** runs of whitespace are collapsed to a single space, and the result is trimmed.

Documents that become empty after this transformation are discarded immediately (this removes the 3 missing abstracts plus any whitespace-only ones).

Shingling. The cleaned text is tokenised on whitespace and represented as the *set* of its word k -shingles, with $k = 5$ (`SHINGLE_SIZE`): a shingle is a contiguous sequence of k tokens, and representing a document by the *set* of shingles it contains (rather than the multiset/sequence) automatically discards repeated internal occurrences, which is the standard construction used to make Jaccard similarity meaningful for near-duplicate detection. For instance, with $k = 3$, the string “the quick brown fox” yields the shingles (**the, quick, brown**) and (**quick, brown, fox**). Documents shorter than k tokens produce an empty shingle set and are discarded at this stage.

Compact shingle hashing. Storing raw token tuples is memory-inefficient and makes set operations (union, intersection, hashing for MinHash) unnecessarily slow. Each shingle tuple is therefore mapped to a deterministic 64-bit integer via the BLAKE2b hash function (8-byte digest) applied to the tuple’s tokens joined by spaces. This is only an internal, memory-saving representation — it is unrelated to, and computed before, the MinHash signature described in Section 5. Using a cryptographic-quality hash function here (rather than Python’s built-in, randomised `hash()`) keeps collision probability negligible and, importantly, keeps results deterministic and reproducible across runs and processes, which matters for the correctness of the subsequent LSH bucketing step.

After hashing, documents whose shingle set is empty are removed. This last filter, combined with the empty-text filter above, reduces the working collection from the 5,000 sampled abstracts to 4,994 non-empty documents, each represented by an average of about 17.5 hashed shingles.

5 Algorithms and implementation

This section describes each algorithm used in the pipeline, its implementation, a brief argument for its correctness, and its time/space complexity; complexities are stated in terms of the number of documents n , the average number of shingles per document m , the number of MinHash hash functions h , and the LSH parameters b (bands) and r (rows per band), with $h = br$.

5.1 Exact Jaccard similarity

For two shingle sets A, B , the exact similarity used throughout the project is the Jaccard index

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|},$$

with the conventions $J(A, B) = 1$ if $A = B = \emptyset$ and $J(A, B) = 0$ if exactly one of A, B is empty. This quantity is computed directly on the hashed shingle sets using Python’s built-in set intersection/union, which is correct by definition and is used in two roles: (i) as the *final verification* applied only to the (small) set of candidate pairs surviving LSH, guaranteeing that no false positive introduced by MinHash/LSH ever reaches the final output, and (ii) as the *ground truth* against which the approximate stages (MinHash, LSH) are validated on a manageable evaluation subset (Section 7).

Computing $J(A, B)$ for one pair costs $O(|A| + |B|) = O(m)$ using hash sets, and comparing every pair of n documents exhaustively therefore costs $O(n^2 m)$ overall — quadratic in the number of documents, which is exactly the cost that the rest of the pipeline is designed to avoid (see Section 6).

5.2 MinHash

Idea and correctness. MinHash produces a fixed-length signature for each document such that the *probability* that two documents agree on one signature component equals their Jaccard similarity. Concretely, for a random permutation π of the universe of all possible shingles, and for a document represented by a shingle set S , define $\text{sig}_\pi(S) = \arg \min_{x \in S} \pi(x)$. For two sets A, B , a standard combinatorial argument shows that

$$\Pr [\text{sig}_\pi(A) = \text{sig}_\pi(B)] = \frac{|A \cap B|}{|A \cup B|} = J(A, B),$$

because the element of $A \cup B$ that is mapped to the smallest value by π is equally likely to be any element of $A \cup B$, and the two signatures agree exactly when that element also belongs to $A \cap B$. Since storing and evaluating a truly random permutation of a huge universe is impractical, the implementation approximates it with a family of $h = 100$ (NUM_HASH_FUNCTIONS) independent hash functions drawn from the universal family

$$h_i(x) = (a_i x + b_i) \bmod p, \quad i = 1, \dots, h,$$

with $p = 4,294,967,311$ a prime larger than the range of the hashed shingle identifiers, and coefficients a_i, b_i drawn uniformly at random once (with a fixed seed, for reproducibility) and reused for every document. For a document with shingle set S , the i -th signature component is

$$\text{sig}_i(S) = \min_{x \in S} h_i(x).$$

Given two signatures, the MinHash-based estimator of Jaccard similarity is the fraction of positions at which they agree,

$$\hat{J}(A, B) = \frac{1}{h} \left| \{ i : \text{sig}_i(A) = \text{sig}_i(B) \} \right|,$$

which, by the argument above and linearity of expectation, is an unbiased estimator of $J(A, B)$ with standard deviation $O(1/\sqrt{h})$: this is the formal justification for using $h = 100$ independent hash functions rather than a single one, and it is validated empirically in Section 7.

Implementation. Algorithm 1 summarises the signature computation. The implementation vectorises the inner loop over all h hash functions simultaneously with NumPy (applying every h_i to every shingle identifier of a document at once, then taking the element-wise minimum with the running signature), avoiding an explicit Python-level loop over hash functions and keeping the per-document cost close to native-speed array operations.

Algorithm 1 MinHash signature computation

Require: shingle set S ; hash coefficients $(a_i, b_i)_{i=1}^h$; prime p

Ensure: signature vector $\text{sig} \in \{0, \dots, p-1\}^h$

- 1: $\text{sig}_i \leftarrow p$ for $i = 1, \dots, h$ ▷ sentinel value
 - 2: **for** each shingle $x \in S$ **do**
 - 3: **for** $i = 1$ to h **do** ▷ vectorised over i in the implementation
 - 4: $v \leftarrow (a_i \cdot x + b_i) \bmod p$
 - 5: $\text{sig}_i \leftarrow \min(\text{sig}_i, v)$
 - 6: **end for**
 - 7: **end for**
 - 8: **return** sig
-

Complexity. Computing one signature costs $O(hm)$ time and $O(h)$ additional space; computing signatures for all n documents therefore costs $O(nhm)$ time and $O(nh)$ space for the resulting signature matrix — *linear* in the number of documents, as opposed to the quadratic cost of exhaustive Jaccard, and independent of how many *other* documents exist.

5.3 Locality-Sensitive Hashing (banding)

Idea and correctness. Even after replacing shingle sets with h -integer signatures, comparing every pair of signatures is still an $O(n^2h)$ operation. LSH avoids this by only ever comparing documents that are likely, but not guaranteed, to be similar, trading a small and controllable probability of missing a similar pair for a large reduction in the number of comparisons performed. Each signature is split into b *bands* of r rows, with $h = br$; two documents are considered a *candidate pair* if they agree on *every* row of *at least one* band. Within a band, the r signature values of a document are combined into a single bucket key using BLAKE2b applied to the raw bytes of the sub-vector (rather than Python’s randomised built-in `hash()`), which is deterministic and makes two documents collide in a band if and only if their r MinHash values in that band are identical — i.e. the bucketing step introduces no false positives or negatives of its own beyond an astronomically unlikely BLAKE2b collision.

For two documents with true Jaccard similarity s , each row is (by the MinHash correctness argument above) an independent Bernoulli trial that agrees with probability s ; the probability that all r rows of a fixed band agree is therefore s^r , and the probability that the band does *not* produce agreement is $1 - s^r$. Since the b bands are constructed from disjoint groups of hash functions and are therefore independent, the probability that *none* of the b bands agrees is $(1 - s^r)^b$, so the probability that the pair becomes a candidate through at least one band is

$$P(\text{candidate} \mid s) = 1 - (1 - s^r)^b,$$

the classical LSH “S-curve”. This formula is what motivates the choice of (b, r) discussed in Section 7: it should be tuned so that its steep transition region is close to the desired similarity threshold.

Implementation. Algorithm 2 summarises candidate generation. For every band, a hash table mapping bucket keys to lists of document indices is built in a single pass over the n documents; any bucket containing at least two documents contributes all $\binom{|\text{bucket}|}{2}$ pairs within it to a global candidate set (implemented as a Python `set` of ordered index pairs, which naturally de-duplicates pairs found in more than one band).

The candidate set C produced by Algorithm 2 is then passed, together with the shingle sets, to the exact-Jaccard verification of Section 5.1: only pairs in C with $J \geq \text{threshold}$ are retained in the final output, which is what guarantees that LSH’s imperfect precision (Section 7) never affects the precision of the *overall* pipeline.

Complexity. Building the hash table for one band costs $O(nr)$ time (one BLAKE2b hash of r integers per document); across b bands, the bucketing itself therefore costs $O(nbr) = O(nh)$ time, i.e. linear in n . The additional cost of enumerating pairs *within* buckets is $O(\sum_{\beta} |\beta|^2)$, which is small in practice whenever the banding parameters are chosen so that most buckets contain only a handful of (mostly dissimilar) documents, as confirmed empirically in Section 6; in the worst case (all documents identical), this term degenerates to $O(n^2)$, but this is not the regime the method targets. Space is $O(nh)$ for the signature matrix plus $O(n)$ per band for the hash table.

Algorithm 2 LSH candidate-pair generation

Require: signature matrix $\text{Sig} \in \mathbb{Z}^{n \times h}$; bands b ; rows per band r ($h = br$)

Ensure: set of candidate pairs C

```
1:  $C \leftarrow \emptyset$ 
2: for band = 0 to  $b - 1$  do
3:   buckets  $\leftarrow$  empty hash table
4:   for doc = 0 to  $n - 1$  do
5:      $k \leftarrow \text{BLAKE2B}(\text{Sig}[\text{doc}, \text{band} \cdot r : (\text{band} + 1) \cdot r])$ 
6:     append doc to buckets[ $k$ ]
7:   end for
8:   for each bucket  $\beta$  in buckets with  $|\beta| \geq 2$  do
9:     add every pair  $(i, j)$ ,  $i < j \in \beta$ , to  $C$ 
10:  end for
11: end for
12: return  $C$ 
```

6 Scalability of the proposed solution

Theoretical scaling. Table 1 summarises the time complexity of each stage derived in Section 5. The key point is that shingling, MinHash and LSH candidate generation are all *linear* in the number of documents n , and only the final verification step is applied to the (empirically small) set of LSH candidates rather than to all $O(n^2)$ pairs; the pipeline as a whole is therefore expected to scale near-linearly with n as long as the number of candidate pairs itself grows sub-quadratically, which is the regime observed empirically below.

Table 1: Time complexity of each pipeline stage (n : documents, m : average shingles/document, $h = br$: MinHash functions, b : bands, r : rows/band, $|C|$: number of LSH candidate pairs).

Stage	Time complexity	Notes
Pre-processing + shingling	$O(nm)$	per-document, embarrassingly parallel
Exhaustive Jaccard (baseline)	$O(n^2m)$	avoided except for verification/ground truth
MinHash signatures	$O(nhm)$	vectorised over h
LSH candidate generation	$O(nh) + O(\sum_{\beta} \beta ^2)$	second term small in practice
Exact verification of candidates	$O(C m)$	$ C \ll \binom{n}{2}$ empirically

Empirical validation. The exhaustive baseline’s quadratic growth is confirmed directly: running times on sub-collections of 100, 250, 500, 750 and 1,000 documents grow from 0.02s to 1.58s (Figure 1), consistent with $O(n^2)$ growth and clearly impractical for the full 16,787-article corpus. In contrast, computing MinHash signatures for all 4,994 documents took only 0.69s, and LSH candidate generation over the same collection took 0.23s and produced only 346 candidates out of $\binom{4994}{2} \approx 1.25 \times 10^7$ possible pairs, a reduction of more than four orders of magnitude in the number of pairs requiring exact verification.

Table 2 reports, for growing sub-collections of the 4,994 pre-processed documents, the number of all possible pairs against the number of LSH candidates and the running time of LSH candidate generation and of exact verification. While the number of possible pairs grows quadratically, the number of candidates — and hence the cost of the only step whose worst case is quadratic — grows far more slowly (Figure 2), and the total running time of the LSH pipeline stays a small fraction of a second even at 3,000 documents (Figure 3). This confirms empirically that the proposed solution scales sub-quadratically in the regime of interest and would remain tractable

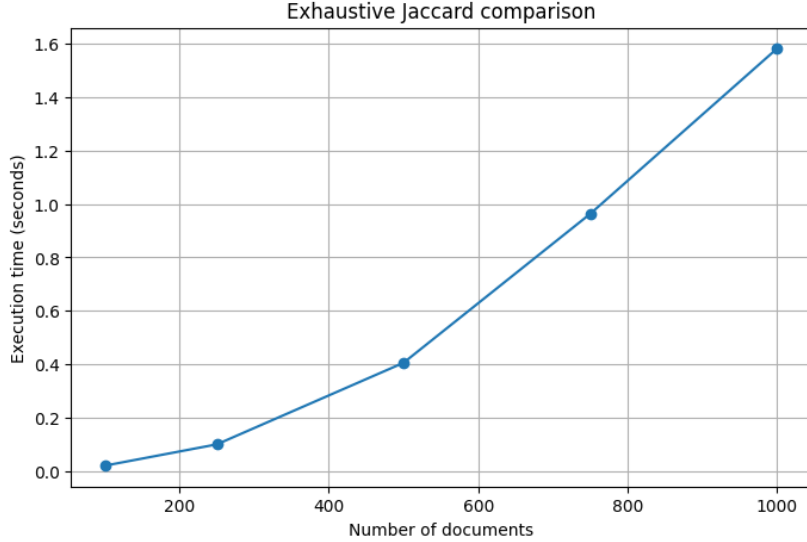


Figure 1: Running time of the exhaustive Jaccard comparison as a function of the number of documents, confirming the expected quadratic growth of the baseline the pipeline is designed to avoid.

on the full dataset, and on collections considerably larger than it, whereas the exhaustive baseline would not.

Table 2: Scalability of the LSH pipeline: possible pairs vs. LSH candidates, and running time (seconds), for growing sub-collections of the 4,994 pre-processed documents.

Documents	Possible pairs	LSH candidates	Verified pairs	LSH time (s)	Total time (s)
500	124,750	1	1	0.019	0.019
1,000	499,500	11	11	0.039	0.039
2,000	1,999,000	67	67	0.078	0.078
3,000	4,498,500	106	103	0.122	0.122

Because exact verification is applied only to LSH candidates, any imprecision of the LSH stage translates only into extra (cheap) verification work, never into a wrong final answer: this is visible in Table 2, where at $n = 3,000$ the 106 candidates include 3 false positives that are correctly filtered out by verification, at negligible extra cost (0.0003s). This design — push the expensive, unbounded computation into a cheap, approximate filtering stage, and reserve the exact, correct computation for a small residual set — is precisely what allows the pipeline to scale to dataset sizes for which the exhaustive baseline is infeasible.

7 Experiments

Three experiments were performed on the pre-processed collection of 4,994 document abstracts, unless a smaller evaluation subset is explicitly stated.

7.1 Accuracy of the MinHash estimator

Goal. Verify empirically that $\hat{J}$ (Section 5.2) is a good estimator of the exact Jaccard similarity J . **Setup.** All $\binom{100}{2} = 4,950$ pairs among the first 100 documents were compared using both the exact Jaccard similarity on their shingle sets and the MinHash estimate on their 100-component

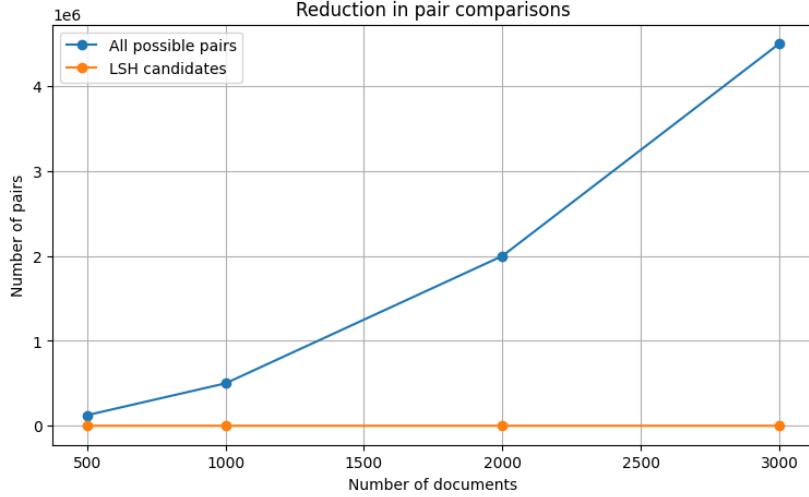


Figure 2: Growth of the number of all possible pairs versus the number of LSH candidate pairs, as the collection size increases.

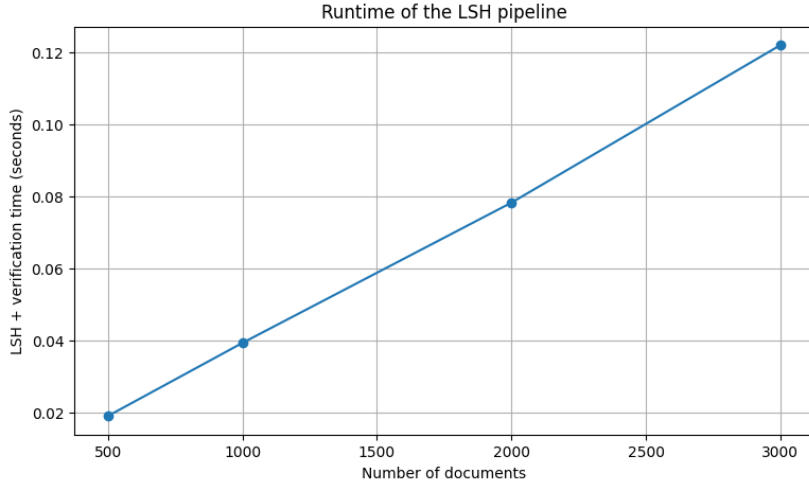


Figure 3: Total running time of the LSH candidate-generation and verification stages as a function of the number of documents.

signatures. **Result.** Figure 4 plots $\hat{J}$ against J for every pair; points lie essentially on the diagonal, and the measured mean absolute error is 0.0000 (to four decimals). This is consistent with the theoretical unbiasedness of $\hat{J}$ (Section 5.2) and is further helped, in this specific collection, by the fact that most abstract pairs share no shingles at all ($J = 0$, correctly estimated as such), while the small number of genuine (near-)duplicates is recovered almost exactly.

7.2 End-to-end detection on the full collection

Goal. Run the complete pipeline (MinHash $\rightarrow$ LSH $\rightarrow$ verification) and inspect the detected near-duplicates. **Setup.** The default configuration ($h = 100$, $b = 20$, $r = 5$, Jaccard threshold 0.50) was applied to all 4,994 documents. **Result.** LSH produced 346 candidate pairs, of which 340 were confirmed similar ($J \geq 0.5$) by exact verification (the remaining 6 were false positives from band-hash collisions, correctly discarded). The highest-scoring detected pairs are exact textual duplicates with $J = 1.000$, and correspond, on inspection, to recurring “template” abstracts that the New York Times reuses verbatim across different articles — e.g. a photo-discussion prompt reused across several “Learning” section pieces, and a recurring live-markets

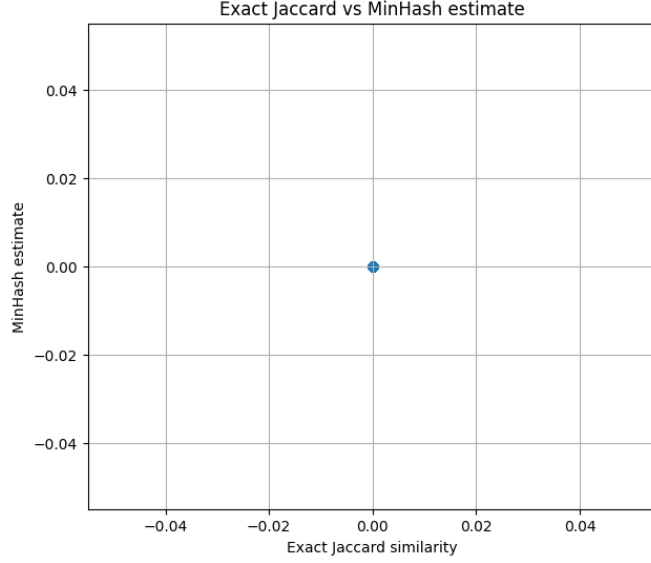


Figure 4: Exact Jaccard similarity versus the MinHash-based estimate, for all pairs among the first 100 documents.

summary used across multiple COVID-19-related articles. This is a sanity check that the pipeline detects genuine, expected near-duplicates rather than spurious matches.

7.3 LSH quality against exact ground truth, and effect of (b, r)

Goal. Quantify how closely LSH approximates exhaustive similarity search, and how this depends on the banding configuration. **Setup.** Exact ground truth was computed on a smaller evaluation subset (the first 1,000 documents, i.e. all 499,500 possible pairs), of which 11 pairs have $J \geq 0.5$. LSH was then run on the same subset for five configurations (b, r) with $b \cdot r = 100$ fixed, and its candidate set was compared against the ground truth using precision, recall and F_1 . **Result.** With the default configuration $(b = 20, r = 5)$, LSH returned exactly the 11 true-positive candidates (precision = 1.0, recall = 1.0, $F_1 = 1.0$), a 100.00% reduction in the number of pairs examined. The full sweep is reported in Table 3 and Figures 5–7.

Table 3: Effect of the LSH banding configuration (b, r) , with $b \cdot r = 100$ fixed, on the number of candidate pairs and on precision/recall/ F_1 against exact ground truth (1,000-document evaluation subset, threshold $J \geq 0.5$).

Bands (b)	Rows/band (r)	Candidates	Precision	Recall	F_1
10	10	7	1.000	0.636	0.778
20	5	11	1.000	1.000	1.000
25	4	11	1.000	1.000	1.000
50	2	21	0.524	1.000	0.688
100	1	63	0.175	1.000	0.297

8 Discussion of results

MinHash accuracy. The near-perfect agreement between $\hat{J}$ and J (Section 7.1) empirically confirms the unbiasedness argument of Section 5.2: with $h = 100$ hash functions, signatures of

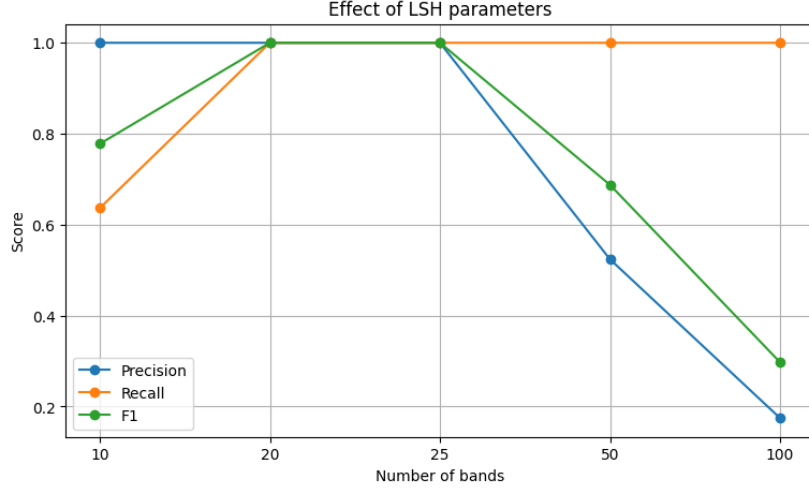


Figure 5: Precision, recall and F_1 of LSH candidate generation as a function of the number of bands b (with $r = 100/b$), against exact ground truth.

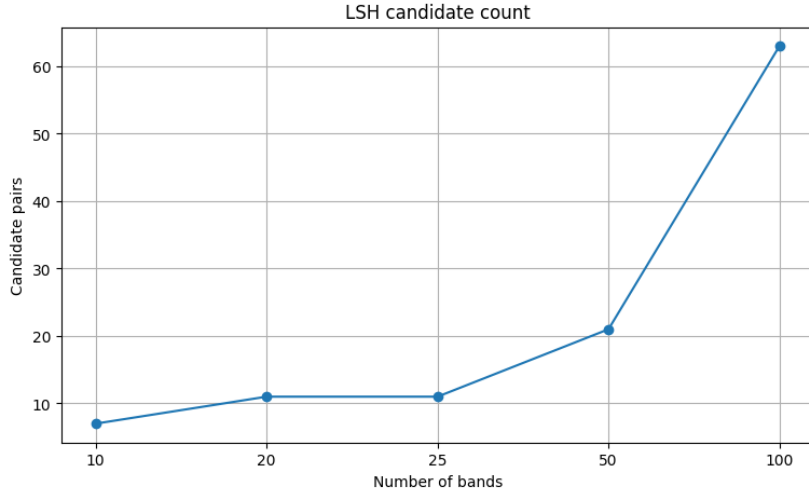


Figure 6: Number of LSH candidate pairs as a function of the number of bands b .

only 100 integers reproduce the exact similarity of 17.5-shingle-average documents essentially exactly, at a fraction of the memory footprint of storing full shingle sets pairwise.

The precision/recall trade-off of LSH banding. Table 3 and Figure 7 exhibit the trade-off predicted by the S-curve $P(\text{candidate} | s) = 1 - (1 - s^r)^b$: with few, “wide” bands (large r , e.g. $b = 10, r = 10$), a genuinely similar pair is unlikely to agree on an entire band by chance, so LSH is conservative and precise (precision 1.0) but misses some true positives (recall drops to 0.636); with many, “narrow” bands (small r , e.g. $b = 100, r = 1$), almost any moderately similar pair shares at least one band, so recall saturates at 1.0 but precision collapses to 0.175, because many more, mostly dissimilar, pairs are also flagged (visible in Figure 6, where candidates grow from 7 to 63 as b grows from 10 to 100). The configuration used by default, $b = 20, r = 5$, sits at the “knee” of this trade-off for the chosen threshold of 0.5: Figure 7 shows that its S-curve has the steepest transition closest to $s = 0.5$ among the five configurations tested, which is exactly why it achieves perfect precision and recall on the evaluation subset while still filtering out effectively all dissimilar pairs. This illustrates the general LSH design principle: (b, r) should be chosen so that the inflection point of the S-curve matches the similarity threshold of interest.

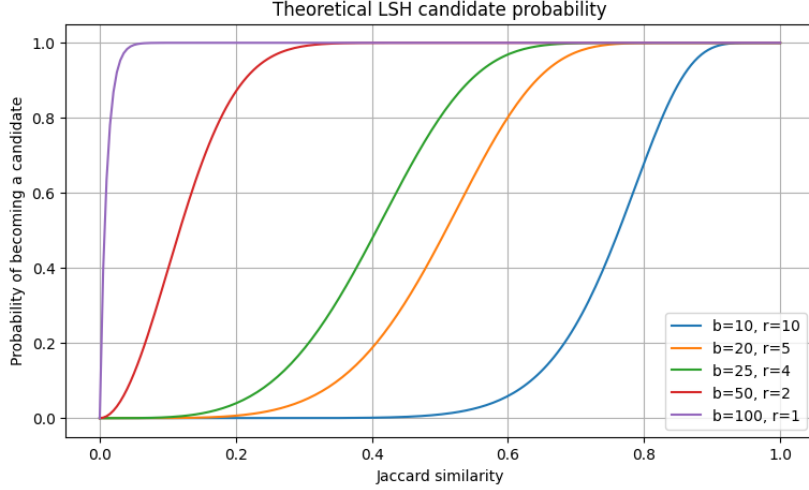


Figure 7: Theoretical probability $P(\text{candidate} \mid s) = 1 - (1 - s^r)^b$ of two documents with true Jaccard similarity s becoming an LSH candidate, for each of the five (b, r) configurations considered.

Why exact verification matters. Because every LSH candidate is re-checked with exact Jaccard, precision losses at the LSH stage (like the 6 false positives out of 346 candidates in Section 7.2, or the 3 out of 106 at $n = 3,000$ in Table 2) never propagate to the final result — they only add a small, bounded amount of extra verification work. Recall losses, in contrast, are irrecoverable: a true positive that LSH fails to place in any shared band is never checked and is silently missed. This asymmetry is the key practical risk of the method and is why recall was evaluated explicitly against ground truth (Section 7.3) rather than inferred from precision alone.

Overall. The combination of MinHash and LSH banding, backed by exact verification, reduces the number of pairwise comparisons required to detect near-duplicate abstracts by two to four orders of magnitude with respect to the exhaustive baseline (Section 6), while, for a well-chosen banding configuration, matching the exhaustive method’s precision and recall exactly on the tested evaluation subset. This makes the approach practical for the full New York Times corpus, and for corpora considerably larger than it, which the exhaustive baseline could not handle.

9 Replicability

To make the experiments reproducible, the notebook fixes every source of randomness and avoids non-deterministic operations in the similarity-critical path:

- A single random seed (`RANDOM_SEED = 42`) is used to seed both Python’s `random` module and NumPy’s random generator, and controls both the document sub-sampling and the MinHash hash-function coefficients (a_i, b_i) .
- Shingle hashing and LSH bucket keys use BLAKE2b, a deterministic hash function, instead of Python’s built-in `hash()`, which is randomised per-process by default and would make bucket assignments (and hence the exact candidate set) non-reproducible across runs.
- All configuration parameters (subsample size, shingle size k , number of hash functions h , bands b and rows r , Jaccard threshold) are declared as named constants in a single configuration cell, so that every number quoted in this report can be regenerated by re-running the notebook top-to-bottom with the same values.

Environment. The notebook is designed to run end-to-end on Google Colab with no manual setup beyond providing Kaggle API credentials: it installs its (few, general-purpose) dependencies at the top of the notebook, authenticates with Kaggle, downloads and extracts the dataset into a scratch directory, and writes its numeric results (comparison tables, parameter sweeps, scalability measurements, detected pairs) to a local `results/` folder for later inspection — the dataset itself is never committed to the repository, only the code and the (small) result tables.

Code availability. The full notebook, together with a README describing how to run it on Google Colab, is available at: **[TODO: insert the public GitHub repository URL here before submission]**.

10 Conclusions

This project implemented, from scratch, a complete pipeline for near-duplicate text detection and evaluated it on abstracts from the New York Times Articles & Comments (2020) dataset. Text pre-processing and k -shingling produce a compact, hashed set representation of each document (Sections 3–4); exact Jaccard similarity provides a correct but quadratic-cost reference (Section 5.1); MinHash compresses each document into a fixed-size signature that provably, and empirically, approximates Jaccard similarity with high accuracy (Section 5.2); LSH banding reduces the number of pairs that must ever be compared from quadratic to a small, tunable candidate set (Section 5.3); and exact verification of the LSH candidates guarantees that the final output has the same precision as the exhaustive method, regardless of any imprecision introduced upstream (Section 6). Empirically, a banding configuration of $b = 20$ bands of $r = 5$ rows (for $h = 100$ MinHash functions) achieves perfect precision and recall against exact ground truth on the tested evaluation subset while reducing the number of pairs to be checked by two to four orders of magnitude, confirming both the correctness and the scalability of the proposed solution with respect to the exhaustive baseline.

A natural extension, not pursued here for time reasons, would be to repeat the full pipeline on the `comments` component of the dataset (which is substantially larger and noisier than the abstracts) and on the full, un-sampled article collection, to further stress-test the scalability claims of Section 6 at a larger scale.

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.